



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Escola Tècnica
Superior d'Enginyeria
Informàtica

Escola Tècnica Superior d'Enginyeria Informàtica
Universitat Politècnica de València

Evolución del Proyecto

Trabajo Fin de Grado

Grado en Informática Industrial y Robótica

Autor: Lourdes Francés Llimerá

Tutor: Juan Francisco Blanes Noguera

2025-2026

Tabla de contenidos

Tabla de contenidos.....	2
Índice de tablas.....	3
1. Propósito de este documento.....	4
2. Punto de partida y objetivo inicial.....	4
3. Elección del framework: ROSA frente a RAI.....	4
3.1. La captura de voz: implementación propia frente a la pila S2S de RAI.....	5
4. El motor de inferencia: del modelo local a la nube	6
4.1. La vía local (Ollama) y sus límites.....	6
4.2. Modelo pequeño frente a modelo grande	6
4.3. Alternativas gratuitas descartadas.....	6
4.4. Decisión: la API de Ollama del ai2 como vía principal y Groq como respaldo	7
5. El entorno de desarrollo condicionado por el hardware	7
6. El giro arquitectónico: de los movimientos reactivos a Nav2	8
7. Localización: de la autolocalización a la pose inicial manual.....	8
8. Cambio de mundo de simulación	9
9. El fallo “Collision Ahead” en arranque en frío	9
10. Del simulador al robot real	10
10.1. Salto de distribución: Humble a Jazzy	10
10.2. La red: dos tarjetas para dos mundos	10
10.3. La altura del LIDAR y el entorno de pruebas	10
11. Modificaciones al framework RAI	11
12. Ajuste del comportamiento del agente (prompt).....	11
13. Síntesis: de dónde se partió y por qué está así.....	12

Índice de tablas

Tabla 1. Resumen de la evolución del proyecto.....	12
--	----

1. Propósito de este documento

Los manuales y la documentación principal de LARIC describen el sistema tal y como es hoy: su arquitectura, sus herramientas y su modo de uso. Este documento es complementario y de naturaleza distinta. Su objetivo es dejar constancia del camino recorrido: de qué punto se partió, qué obstáculos fueron apareciendo, qué alternativas se valoraron en cada bifurcación y por qué se tomó una decisión y no otra.

La intención es doble. Por un lado, justificar las decisiones de diseño que de otro modo podrían parecer arbitrarias (por ejemplo, por qué todo el movimiento pasa por Nav2, o por qué la localización inicial es manual). Por otro, documentar las vías que se intentaron y se descartaron, de modo que ese trabajo no quede invisible y no se repitan callejones sin salida ya explorados. El relato sigue, en líneas generales, el orden cronológico en que se afrontaron los problemas.

2. Punto de partida y objetivo inicial

El objetivo de partida era controlar un robot móvil mediante lenguaje natural, por voz y por texto, sin que el usuario necesitara conocimientos de robótica. La idea concreta era un sistema de tipo Push-to-Talk, con una interfaz que permitiera saber en todo momento cuándo está hablando el usuario, qué ha entendido el robot y qué va a ejecutar, además de una parada de emergencia.

Sobre esa base, el primer prototipo funcional fue una interfaz sencilla que capturaba la voz y la transcribía correctamente. Validado ese punto de entrada, el trabajo se centró en elegir las herramientas que conectarían el lenguaje natural con la acción del robot, y en construir progresivamente el repertorio de comandos.

3. Elección del framework: ROSA frente a RAI

La primera decisión de peso fue qué framework emplear como puente entre el modelo de lenguaje y ROS 2. Se estudiaron dos candidatos: ROSA y RAI (Robotec AI Framework). El análisis mostró que ambas herramientas son, en el fondo, muy parecidas: las dos se apoyan en un LLM con herramientas (tools) y system prompts, y comparten la misma estructura conceptual de agente. Para entender bien esa estructura fue necesario investigar previamente varios conceptos (ReAct, LLM, LangChain, tools, system prompts), lo que retrasó la decisión pero permitió compararlas con criterio.

La balanza se inclinó por RAI por tres motivos prácticos:

- **Integración ROS 2 – LLM ya construida:** RAI aporta resuelto el puente entre el modelo de lenguaje y ROS 2 (ROS2Context, ROS2Connector y

herramientas preparadas para Nav2), evitando tener que programar esa capa desde cero.

- **Ejemplos con robots móviles (AMR):** RAI aporta ejemplos directamente aplicables a robots móviles autónomos y a control por voz, más cercanos al objetivo del TFG.
- **Documentación más extensa:** Facilita encontrar ejemplos de referencia durante el desarrollo del código.

En la valoración inicial se consideró además una ventaja que RAI integrase nodos de ROS 2 para gestionar el micrófono y la transcripción. En la práctica, sin embargo, la captura de voz y la transcripción se acabaron implementando de forma propia en la interfaz (Push-to-Talk con sounddevice y la API de Google Speech Recognition), por lo que esa capacidad concreta de RAI no llegó a emplearse. Las ventajas que realmente pesaron fueron, por tanto, la integración con ROS 2 ya resuelta, los ejemplos de AMR y el soporte documental. Esta elección condiciona el resto del proyecto: toda la arquitectura posterior se construye sobre los mecanismos de RAI (ROS2Context, ROS2Connector y sus herramientas).

3.1. La captura de voz: implementación propia frente a la pila S2S de RAI

Que LARIC no utilice la pila de voz de RAI es una decisión de diseño coherente con el resto del sistema. La captura y la transcripción de voz se implementaron de forma propia en la interfaz: Push-to-Talk (barra espaciadora) con sounddevice para grabar y la API de Google Speech Recognition para transcribir, con conmutación de idioma (es-ES / en-US). De igual forma, para dotar al sistema de respuestas habladas, se incorporó la salida por voz empleando edge-tts. RAI, por su parte, ofrece una pila completa de speech-to-speech (rai_s2s) con detección de actividad de voz (VAD), modelos de transcripción locales de tipo Whisper y síntesis de voz (TTS).

Se optó por la solución propia por tres razones alineadas con decisiones ya tomadas en el proyecto:

- **Coste computacional:** La transcripción local de RAI (modelos Whisper) exige cómputo en la propia máquina, justo lo que se evitó al trasladar la inferencia del LLM a la nube o al servidor del laboratorio. La transcripción de Google STT y la síntesis con edge-tts no consumen CPU/GPU local, lo que mantiene la VM libre para Gazebo, ROS 2 y Nav2.
- **Modelo de interacción más seguro:** El Push-to-Talk es una activación explícita (el usuario decide cuándo habla) frente a la escucha continua (VAD) de RAI. En un sistema que mueve un robot físico, evitar que el habla ambiental dispare acciones es una ventaja, y además cumple el requisito de saber en todo momento cuándo está hablando el usuario.
- **Menor superficie de dependencia:** Habiendo sido ya necesario corregir RAI en la capa de navegación (sección 11), adoptar otro de sus subsistemas habría añadido complejidad e incertidumbre a un componente que ya funcionaba bien.

La decisión tiene contrapartidas que conviene reconocer. La API de Google Speech Recognition empleada es un punto de acceso gratuito y no oficial: requiere conexión a Internet, no ofrece garantías de servicio y envía el audio a un tercero. Para el alcance del TFG resulta suficiente, pero queda como trabajo futuro sustituir el motor de STT y TTS por una API oficial o por un modelo local. En ese escenario (o si se requiriese operación sin Internet o palabra de activación), la pila S2S de RAI sería la opción preferible.

4. El motor de inferencia: del modelo local a la nube

Quizá el obstáculo más persistente de todo el desarrollo fue elegir dónde y cómo ejecutar el LLM. La premisa inicial era no depender de Internet y ejecutar el modelo en local. Esa vía chocó enseguida con la realidad del hardware disponible.

4.1. La vía local (Ollama) y sus límites

Al probar modelos en local con Ollama aparecieron dos extremos igual de inviables. Los modelos grandes (20 GB o más) no cabían con holgura en el entorno de desarrollo; los pequeños (en torno a 5 GB) sí cabían pero se quedaban colgados al analizar las instrucciones. El problema de fondo es que ejecutar un LLM en local exige mucha CPU/RAM, y debía convivir además con el simulador (Gazebo) y ROS 2 ejecutándose a la vez.

4.2. Modelo pequeño frente a modelo grande

Trasladada la inferencia a la nube, la comparación entre tamaños de modelo fue concluyente:

- **Llama 3.1 8B (gratuito):** Ligero, pero le falta capacidad de razonamiento. Fallaba al procesar las reglas de seguridad y llegaba a confundir signos positivos y negativos (clave para ángulos y distancias), con una latencia media de unos 5 segundos.
- **Llama 3.3 70B:** Funciona correctamente y razona bien las reglas, pero la capa gratuita de Groq impone un tope de 100.000 tokens al día. Como en cada comando se envían por debajo el system prompt y las herramientas, la cuota se agotaba en 60-80 pruebas, paralizando el desarrollo hasta el día siguiente.

4.3. Alternativas gratuitas descartadas

Antes de plantear un modelo de pago se exploraron otras opciones gratuitas, todas con bloqueos:

- **Google Gemini (API):** Ofrece un modelo flash con límites gratuitos muy altos, pero por la regulación europea Google mantiene bloqueada la capa gratuita en España y exige una cuenta de facturación.

- **Modelos en local (Ollama):** Descartados de nuevo por la exigencia de CPU/RAM ya comentada.

4.4. Decisión: la API de Ollama del ai2 como vía principal y Groq como respaldo

La solución que resolvió el problema de raíz fue apoyarse en la infraestructura de inferencia del instituto ai2 de la UPV. El ai2 mantiene varios modelos LLM ya desplegados en sus servidores (entre ellos Llama 3.3 70B) accesibles mediante una API de Ollama alojada en su clúster Kubernetes. El acceso requiere autenticación y estar conectado a la red de la UPV. Esta es la vía principal del sistema porque no tiene límite de tokens.

Para poder seguir trabajando fuera de la universidad, donde no hay acceso a esa red, el sistema recurre como respaldo a la capa gratuita de Groq sobre el mismo modelo (Llama 3.3 70B). Esta vía secundaria sí arrastra el límite de 100.000 tokens diarios, asumible por aplicarse solo a las sesiones en remoto.

Precisamente porque la vía principal (la API del ai2) ya ofrece inferencia de 70B sin límite, no fue necesario contratar la modalidad de pago por uso (On-Demand) de Groq. Llegó a evaluarse (su coste habría sido muy bajo, del orden de 0,59 \$ por millón de tokens y un gasto total inferior a unos pocos euros, dado que cada prueba consume 1.500-2.000 token), pero resultaba simplemente innecesaria: Groq solo actúa como comodín desde casa.

De aquí nace el conmutador `is_from_lab` de `config.py`: a `True` (valor por defecto) dirige las consultas a la API de Ollama del ai2 (vía principal, sin límite de tokens, pero solo desde la red de la UPV); a `False` usa la capa gratuita de Groq en la nube (respaldo desde cualquier sitio, a cambio del tope diario). En el código, el primer caso instancia `ChatOllama` (apuntando a la URL del clúster) y el segundo `ChatGroq`. El sistema queda así desacoplado de un único proveedor de inferencia, que era justamente el punto más frágil al principio.

5. El entorno de desarrollo condicionado por el hardware

Una parte de las decisiones de infraestructura no responde a preferencias técnicas sino al hardware del que se disponía. No se contaba con un equipo con Ubuntu nativo dedicado al proyecto, por lo que el desarrollo se realizó sobre una máquina virtual (VMware con Ubuntu 22.04). Esta elección, tomada para evitar reinstalaciones y poder trabajar con snapshots restaurables, marca varias consecuencias posteriores.

- **ROS 2 Humble sobre Ubuntu 22.04 para la simulación:** Se eligió Humble por los problemas de rendimiento de Gazebo bajo Ubuntu 24.04 y con la aceleración gráfica de la máquina virtual.
- **Aceleración 3D obligatoria:** Sin activar “Accelerate 3D graphics” en la VM, RViz no arranca correctamente.

Este condicionante del hardware reaparecerá más adelante, de forma todavía más determinante, al conectar el robot real (sección 10).

6. El giro arquitectónico: de los movimientos reactivos a Nav2

La primera versión funcional del agente se construyó con herramientas de movimiento de bajo nivel de carácter reactivo: avanzar y retroceder a velocidad baja, media o alta (por tiempo, de forma indefinida o por una distancia indicada), girar de manera análoga por tiempo o por ángulo, explorar el entorno de forma autónoma y detectar obstáculos por delante y por detrás para frenar automáticamente. El cálculo de distancias y ángulos se hacía de forma aproximada a partir de la odometría.

Este enfoque funcionaba en entornos sencillos y controlados, pero operaba “a ciegas” desde el punto de vista de la localización: el robot ejecutaba movimientos relativos sin conocer su posición en un mapa. La conclusión, central para el proyecto, fue que los robots no trabajan en modo reactivo: para seguir consignas de control de posición necesitan conocer su posición en el espacio en todo momento.

En consecuencia, se refactorizó toda la capa de movimiento para delegarla en Nav2. Las herramientas pasaron a apoyarse en los Behavior Plugins de Nav2 (`/spin` para los giros, `/drive_on_heading` para los avances y `/backup` para los retrocesos) y en `/navigate_to_pose` para la navegación a destinos. Sobre esta misma base arquitectónica se fundamenta la herramienta de navegación secuencial (`navigate_sequence`), que dota al sistema de la capacidad de recorrer en orden varias ubicaciones conocidas del mapa. A diferencia de rutinas como `execute_sequence` (destinadas a encadenar movimientos primitivos locales), la navegación secuencial permite crear rutas de patrulla o vigilancia, deteniéndose unos segundos en cada punto objetivo antes de reanudar el recorrido global. Con ello todo el control de movimiento opera dentro del marco de navegación, con la posición del robot siempre conocida. El precio asumido es que todo movimiento exige una localización previa, lo que conduce directamente al siguiente obstáculo.

7. Localización: de la autolocalización a la pose inicial manual

Adoptado Nav2, surgió la cuestión de cómo establecer la posición inicial del robot en el mapa. La primera aproximación fue una autolocalización: forzar la localización global de AMCL inyectando una covarianza muy alta y hacer girar el robot 360°, de modo que el LIDAR escanease las paredes y las partículas convergieran.

Esta técnica funcionaba en el primer mundo de pruebas, con elementos muy diferenciados, pero fallaba en un entorno simétrico como una casa de habitaciones cuadradas y poco amuebladas: al ver el LIDAR solo líneas 2D, AMCL confundía

estancias de geometría similar y el robot creía estar en un sitio distinto del real, lo que después bloqueaba la navegación. Se llegó a diseñar una solución de localización asistida por el usuario (que el agente preguntara la sala ante una covarianza alta), pero se descartó.

La decisión adoptada fue la práctica habitual en robótica con un robot real: no dejar la pose inicial en manos del robot ni de la IA, por el riesgo de que no encuentre correspondencia con el entorno, sino fijarla de la forma más precisa posible mediante intervención del operador. En simulación se hace con la herramienta “2D Pose Estimate” de RViz (que publica en `/initialpose`); en el robot real, de forma equivalente, desde Foxglove. La localización pasó así a ser un requisito previo explícito: ninguna herramienta de movimiento se ejecuta sin que el operador haya fijado antes la pose (primitiva `localised_event`).

8. Cambio de mundo de simulación

El desarrollo inicial se hizo sobre el mundo de pruebas básico de TurtleBot3. Para acercar la simulación a un escenario realista de interiores se pasó al mundo `turtlebot3_house`. Este cambio no fue inmediato: al no ser un entorno con elementos diferenciados, destapó precisamente el problema de localización descrito en la sección anterior y obligó a consolidar la inicialización manual de la pose antes de poder trabajar con él de forma estable.

9. El fallo “Collision Ahead” en arranque en frío

Ya con Nav2 y el mapa de la casa, apareció un fallo llamativo: al ejecutar un movimiento primitivo (`/spin`, `/drive_on_heading` o `/backup`) sin haber ejecutado antes una acción de navegación, Nav2 abortaba el movimiento por “Collision Ahead” (colisión inminente), pese a haber espacio libre de sobra. El origen eran partículas fantasma en el `costmap` al arrancar en frío.

La depuración fue larga y se ensayaron varias vías sin éxito (entre ellas, convertir los movimientos en `/navigate_to_pose` o introducir un giro de calibración inicial). La pista decisiva llegó por una asimetría curiosa: girar a la izquierda y avanzar abortaba, pero girar a la derecha y avanzar no. La diferencia era que un giro dejaba el robot de frente al muro y el otro de espaldas. Eso apuntó a que el problema lo causaba tener el muro muy cerca al aparecer (`spawn`), aunque hubiera holgura para moverse.

Se valoraron tres soluciones: (1) dejar el fallo documentado, exigiendo una navegación previa; (2) reconvertir los movimientos simples en navegación, perdiendo el matiz de “detenerse ante un obstáculo” en favor de “rodearlo”; y (3) eliminar los movimientos simples. Finalmente se confirmó que el fallo provenía del punto de aparición del robot pegado a la pared, y bastó con alejar el `spawn` del muro para que todos los comandos funcionaran sin calibración previa. Se trata, por tanto, de un artefacto de la simulación (Nav2 + Gazebo + posición de `spawn`) y no de un defecto del sistema; se ha comprobado que no se reproduce en el robot real.

10. Del simulador al robot real

El traslado a la plataforma física (un Husarion ROSbot 3) introdujo dos obstáculos importantes, uno de versiones y otro de red, ambos muy condicionados por el hardware disponible.

10.1. Salto de distribución: Humble a Jazzy

El ROSbot 3 funciona con ROS 2 Jazzy, y el cliente debe coincidir con la distribución del robot. Esto obligó a preparar un entorno con Ubuntu 24.04 + Jazzy para el lado cliente, distinto del entorno de simulación (Humble), manteniendo el mismo código del paquete `laric_core` y diferenciando los entornos mediante los conmutadores de `config.py`.

10.2. La red: dos tarjetas para dos mundos

El obstáculo de red nace de una restricción del entorno: la red de la universidad (UPVNET) exige credenciales, por lo que el robot no puede conectarse directamente a ella, y se necesita una red local para el tráfico ROS 2. Pero el sistema necesita, a la vez, acceso a Internet para alcanzar la API del LLM. Ambas necesidades chocaban: conectar el robot a la red local dejaba el conjunto sin Internet.

La solución aprovecha que la máquina virtual puede tener dos tarjetas de red:

- **Tarjeta 1 (WiFi → UPVNET):** Da acceso a la API del LLM (Groq) y al servidor Ollama del laboratorio.
- **Tarjeta 2 (Ethernet → router local MERCUSYS_ai2, sin Internet):** Red local a la que el ROSbot se conecta por WiFi y por la que circula todo el tráfico ROS 2 (DDS) entre la VM y el robot (el robot tiene la IP 192.168.1.46).

Quedaba un obstáculo adicional: el arranque de la navegación del Husarion intentaba una descarga (pull) por Internet en cada ejecución, lo que fallaba en la red local aislada (“network is unreachable”). Se resolvió modificando el snap de navegación para que no realice esa descarga en cada arranque, de modo que funcione sin salida a Internet. Conviene subrayar que, con esta configuración, la conectividad no es una limitación del sistema: el PC mantiene Internet por WiFi y, a la vez, alcanza el robot por el router local vía Ethernet.

10.3. La altura del LIDAR y el entorno de pruebas

En las pruebas en el laboratorio se observó una limitación física: el LIDAR del ROSbot 3 no detecta las patas y ruedas de las sillas, los pies de la pizarra ni los pies de las personas, por lo que el robot tiende a chocar con esos obstáculos “invisibles”; además, avanzaba muy despacio en pasillos estrechos pese a tener holgura. Por ello se decidió trasladar los recorridos al exterior del laboratorio y a los corredores de la planta, y el mapa semántico del robot real recoge waypoints de pasillos y zonas abiertas en lugar de puntos interiores del laboratorio.

11. Modificaciones al framework RAI

Durante la integración de la navegación (`navigate_to_location`) se detectaron dos defectos en las herramientas de Nav2 de RAI. Ambos residen en un único archivo (`tools/ros2/navigation/nav2.py`) y afectan a cómo RAI expone el resultado de una acción `/navigate_to_pose`, que LARIC consulta por sondeo para saber si el robot ha llegado o ha fallado. Inicialmente se valoró puentear los fallos desde el código de LARIC, pero se optó por la vía más correcta: corregirlos en el código fuente, sobre un fork propio de RAI. Ahora bien, esta clase de problema (que el estado de una acción previa se “colase” en la siguiente) no era exclusivo de la navegación: también afectaba a los movimientos primitivos (`/spin`, `/drive_on_heading`, `/backup`), donde, tras una cancelación, el siguiente comando podía heredar el estado “cancelado” del anterior y abortarse a sí mismo nada más empezar. La diferencia está en dónde se corrige cada caso: en los primitivos, LARIC reinicia en su propio código el evento compartido `abort_event` al inicio de cada comando; en la navegación, ese estado lo mantiene RAI internamente y por eso hubo que corregirlo en el fork, como se detalla a continuación.

- **Estado de la navegación no reiniciado entre misiones:** Las herramientas de RAI guardan el progreso y el resultado de la acción en variables compartidas a nivel de módulo (`current_action_id`, `current_feedback`, `current_result`). El código original solo las rellena desde los callbacks del objetivo en curso, pero nunca las limpia. En consecuencia, tras la primera navegación, la herramienta de resultado seguía devolviendo indefinidamente el estado terminal de la misión anterior: la siguiente navegación se daba por concluida nada más empezar, al leer ese estado obsoleto. La corrección reinicia esas variables a `None` al comienzo de cada `NavigateToPoseTool`, de modo que cada misión parte de un estado limpio y espera a su propio `Result`.
- **GoalStatus descartado en el resultado:** La herramienta de resultado devolvía `str(current_result.result().result)`, es decir, el mensaje `Result` de `NavigateToPose` (vacío en Nav2), descartando el `GoalStatus`. Así devolvía lo mismo tanto si el robot llegaba como si Nav2 abortaba. La corrección pasa a devolver `str(current_result.result().status)`, el `GoalStatus` real (4 = `SUCCEEDED`, 5 = `CANCELED`, 6 = `ABORTED`), que `NavigationTool` compara para distinguir una llegada de un fallo.

Ambas correcciones son imprescindibles para LARIC: sin la primera, solo la primera navegación de cada sesión funcionaría; sin la segunda, nunca se distinguiría una llegada de un fallo. Esta dependencia del fork debe tenerse en cuenta: con el RAI oficial, la navegación no detectaría correctamente su resultado.

12. Ajuste del comportamiento del agente (prompt)

Un trabajo transversal a todas las fases, y a menudo subestimado, fue el ajuste del system prompt. En varias ocasiones el agente entendía la intención pero pasaba mal los parámetros (signos de ángulo, magnitudes), o encadenaba acciones de forma indebida, o entraba en bucles al recibir errores de Nav2. Buena parte del esfuerzo de depuración consistió en afinar las reglas del prompt: convenciones de signo, una sola herramienta por turno, prohibición de auto-cancelarse, gestión de la asincronía de Nav2 y regla de idioma. El SYSTEM_PROMPT es, de hecho, la pieza que convierte un comportamiento probabilístico en uno suficientemente determinista para controlar un robot con seguridad.

13. Síntesis: de dónde se partió y por qué está así

La tabla siguiente resume el recorrido: el punto de partida de cada decisión, el obstáculo que obligó a replantearlo y la solución que define el estado actual del sistema.

Aspecto	Punto de partida	Obstáculo	Estado actual
Framework	ROSA o RAI (similares)	Necesidad de buenos ejemplos de AMR	RAI
Inferencia LLM	Modelo local sin Internet	Modelos grandes no caben; pequeños no razonan; límites/bloqueos gratuitos	API de Ollama del ai2 (principal) o Groq gratuito desde casa
Tamaño del modelo	Llama 3.1 8B (gratuito)	Confunde signos y falla reglas de seguridad	Llama 3.3 70B
Entorno de desarrollo	Ubuntu nativo (no disponible)	Sin equipo dedicado; Gazebo en 24.04/VM	VM VMware + Ubuntu 22.04 + Humble
Movimiento	Herramientas reactivas por odometría	El robot opera “a ciegas”, sin posición en el mapa	Todo el movimiento por Nav2 (Behavior Plugins)
Localización inicial	Autolocalización (giro 360°)	Falla en entornos simétricos (la casa)	Pose manual
Mundo de simulación	Mundo básico de TurtleBot3	Poco realista para interiores	turtlebot3_house
Robot real	Mismo entorno que la simulación	El ROSbot usa Jazzy; UPVNET exige credenciales	Cliente Ubuntu 24.04 + Jazzy; red con doble NIC
Resultado de navegación	Herramientas de RAI sin modificar	Estado no reiniciado entre misiones; GoalStatus descartado	Fork de RAI (nav2.py): reinicio de estado y GoalStatus real

Tabla 1. Resumen de la evolución del proyecto.

El hilo conductor de todas estas decisiones es la convergencia hacia un diseño robusto y portable: un agente cuya lógica no cambia entre simulación y robot real, que delega el movimiento en un stack de navegación probado, que no depende de un único proveedor de inferencia y cuyas decisiones de infraestructura responden, en buena parte, a las restricciones reales de hardware y de red bajo las que se desarrolló el trabajo.